# HCL TECH- CLOUD & DATA ENGINEERING

| | |
|---|---|
| **Name** | Adarsh Hajgulkar |
| **USN / Roll Number** | 2JI23CS005 |
| **College Name** | Jain College of Engineering, Belagavi |
| **Batch** | 2023 - 2027 |
| **Date** | 28 oct 2026 |
| **Title** | HCLTECH - CLOUD & DATA ENGINEERING ASSESSMENT 2 |

# Question 1: Database Design and Advanced Queries (8 Marks)

```sql
create database EnterpriseDB;
use EnterpriseDB;

CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50) NOT NULL UNIQUE,
    Location VARCHAR(50) NOT NULL
);
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(100) NOT NULL,
    DepartmentID INT,
    Salary DECIMAL(10,2) NOT NULL CHECK (Salary > 0),
    JoiningDate DATE NOT NULL,
    ManagerID INT,
    FOREIGN KEY (DepartmentID)
        REFERENCES Departments(DepartmentID),
    FOREIGN KEY (ManagerID)
        REFERENCES Employees(EmployeeID)
);
```

Ln 30, Col 27

Output | Result #1

| DepartmentID | DepartmentName | Location |
|---|---|---|
| 1 | IT | Bengaluru |
| 2 | HR | Belagavi |
| 3 | Finance | Mumbai |
| 4 | Marketing | Pune |
| 5 | Research | Mysuru |

OK, 5 records retrieved in 1.282ms

MySQL Workbench

```sql
INSERT INTO Departments
(DepartmentID, DepartmentName, Location)
VALUES
(1, 'IT', 'Bengaluru'),
(2, 'HR', 'Belagavi'),
(3, 'Finance', 'Mumbai'),
(4, 'Marketing', 'Pune'),
(5, 'Research', 'Mysuru');

SELECT * FROM Departments;

INSERT INTO Employees
(EmployeeID, EmployeeName, DepartmentID, Salary, JoiningDate, ManagerID)
VALUES
(101, 'Rahul', 1, 90000, '2020-01-15', NULL),
(102, 'Priya', 1, 65000, '2021-03-10', 101),
(103, 'Amit', 1, 55000, '2022-06-20', 101),
(104, 'Sneha', 1, 48000, '2023-02-12', 102),
(105, 'Kiran', 1, 52000, '2022-09-05', 102),
(106, 'Neha', 2, 70000, '2019-07-18', 101),
(107, 'Rohit', 2, 40000, '2023-04-01', 106),
(108, 'Pooja', 2, 45000, '2022-11-11', 106),
(109, 'Vikas', 3, 85000, '2020-08-25', 101),
(110, 'Anjali', 3, 60000, '2021-12-14', 109),
(111, 'Suresh', 3, 50000, '2023-05-08', 109),
```

Ln 34, Col 7

Output | Result #1

| EmployeeName | DepartmentName | ManagerName |
|---|---|---|
| Rahul | IT | NULL |
| Priya | IT | Rahul |
| Amit | IT | Rahul |
| Sneha | IT | Priya |
| Kiran | IT | Priya |

# task:

```sql

    -- tasks (a)
SELECT e.EmployeeName,d.DepartmentName,
       m.EmployeeName AS ManagerName
FROM Employees e
LEFT JOIN Departments d
    ON e.DepartmentID = d.DepartmentID
LEFT JOIN Employees m
    ON e.ManagerID = m.EmployeeID;

    -- (b)
SELECT EmployeeName, DepartmentID, Salary
FROM Employees e
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employees
    WHERE DepartmentID = e.DepartmentID
);

    -- (c)
SELECT MAX(Salary) AS SecondHighestSalary
FROM Employees
WHERE Salary < (
    SELECT MAX(Salary)
    FROM Employees
);
```

```sql
-- (d)
SELECT EmployeeID, EmployeeName, DepartmentID, Salary, SalaryRank
FROM (
    SELECT EmployeeID, EmployeeName, DepartmentID, Salary,
           RANK() OVER (
               PARTITION BY DepartmentID
               ORDER BY Salary DESC
           ) AS SalaryRank
    FROM Employees
) AS RankedEmployees
WHERE SalaryRank <= 3;
```

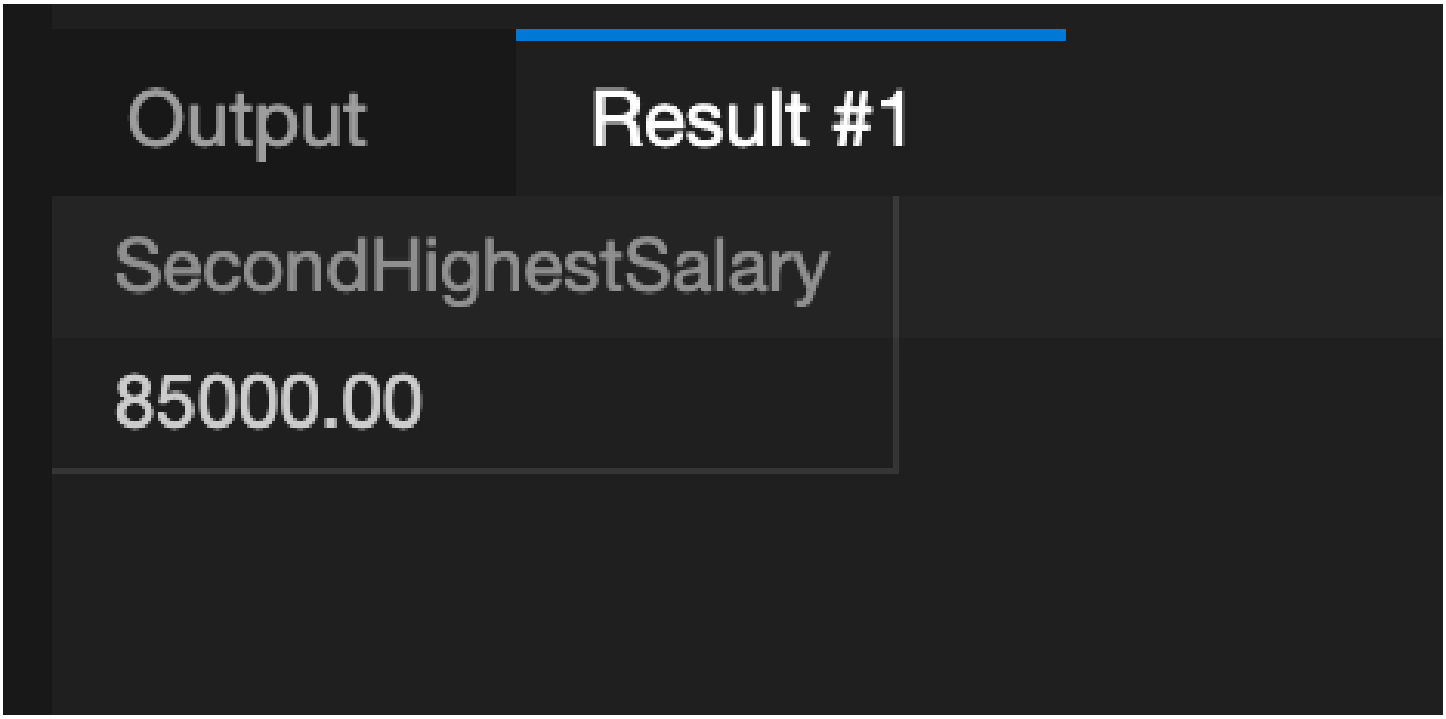
**outputs :**

Output | Result #1

| EmployeeName | DepartmentID | Salary |
|---|---|---|
| Rahul | 1 | 90000.00 |
| Priya | 1 | 65000.00 |
| Neha | 2 | 70000.00 |
| Vikas | 3 | 85000.00 |
| Meera | 4 | 75000.00 |

Output | Result #1

| EmployeeName | DepartmentID | Salary |
|---|---|---|
| Rahul | 1 | 90000.00 |
| Priya | 1 | 65000.00 |
| Neha | 2 | 70000.00 |
| Vikas | 3 | 85000.00 |
| Meera | 4 | 75000.00 |




Output | Result #1

| EmployeeID | EmployeeName | DepartmentID | Salary | SalaryRank |
|---|---|---|---|---|
| 101 | Rahul | 1 | 90000.00 | 1 |
| 102 | Priya | 1 | 65000.00 | 2 |
| 115 | Manoj | 1 | 58000.00 | 3 |
| 106 | Neha | 2 | 70000.00 | 1 |
| 108 | Pooja | 2 | 45000.00 | 2 |
| 107 | Rohit | 2 | 40000.00 | 3 |
| 109 | Vikas | 3 | 85000.00 | 1 |
| 110 | Anjali | 3 | 60000.00 | 2 |
| 111 | Suresh | 3 | 50000.00 | 3 |
| 112 | Meera | 4 | 75000.00 | 1 |

## Question 2: Sales Analytics and Query Optimization (6 Marks)

```sql
CREATE TABLE Sales (
    SaleID INT PRIMARY KEY,
    CustomerID INT,
    SaleDate DATE,
    ProductName VARCHAR(50),
    Quantity INT,
    UnitPrice DECIMAL(10,2)
);

INSERT INTO Sales VALUES
(1, 101, '2025-01-10', 'Laptop', 2, 50000),
(2, 102, '2025-01-15', 'Mouse', 5, 500),
(3, 101, '2025-01-20', 'Keyboard', 3, 1000),
(4, 103, '2025-02-05', 'Laptop', 1, 50000),
(5, 104, '2025-02-12', 'Mouse', 10, 500),
(6, 102, '2025-02-18', 'Keyboard', 2, 1000),
(7, 105, '2025-03-03', 'Laptop', 3, 50000),
(8, 101, '2025-03-11', 'Mouse', 4, 500),
(9, 103, '2025-03-20', 'Keyboard', 5, 1000),
(10, 104, '2025-04-02', 'Laptop', 2, 50000),
```

Output | Result #1

| SaleID | CustomerID | SaleDate | ProductName | Quantity | UnitPrice |
|---|---|---|---|---|---|
| 1 | 101 | 01/10/2025 | Laptop | 2 | 50000.00 |
| 2 | 102 | 01/15/2025 | Mouse | 5 | 500.00 |
| 3 | 101 | 01/20/2025 | Keyboard | 3 | 1000.00 |
| 4 | 103 | 02/05/2025 | Laptop | 1 | 50000.00 |
| 5 | 104 | 02/12/2025 | Mouse | 10 | 500.00 |
| 6 | 102 | 02/18/2025 | Keyboard | 2 | 1000.00 |
| 7 | 105 | 03/03/2025 | Laptop | 3 | 50000.00 |
| 8 | 101 | 03/11/2025 | Mouse | 4 | 500.00 |
| 9 | 103 | 03/20/2025 | Keyboard | 5 | 1000.00 |
| 10 | 104 | 04/02/2025 | Laptop | 2 | 50000.00 |

## task:

```sql

-- task (a)
SELECT ProductName,
       SUM(Quantity * UnitPrice) AS TotalRevenue
FROM Sales
GROUP BY ProductName;

-- (b)
SELECT Month,
       Revenue,
       Revenue - LAG(Revenue) OVER (ORDER BY Month) AS RevenueChange
FROM (
    SELECT DATE_FORMAT(SaleDate, '%Y-%m') AS Month,
           SUM(Quantity * UnitPrice) AS Revenue
    FROM Sales
    GROUP BY DATE_FORMAT(SaleDate, '%Y-%m')
) AS M
ORDER BY Month;
-- (c)
SELECT *
FROM Employees
WHERE JoiningDate >= '2025-01-01'
  AND JoiningDate < '2026-01-01';

```

Output | Result #1

| Month | Revenue | RevenueChange |
|---|---|---|
| 2025-01 | 105500.00 | NULL |
| 2025-02 | 57000.00 | -48500.00 |
| 2025-03 | 157000.00 | 100000.00 |
| 2025-04 | 108000.00 | -49000.00 |
| 2025-05 | 56000.00 | -52000.00 |

Output Result #1

| Month | Revenue | RevenueChange |
|---|---|---|
| 2025-01 | 105500.00 | NULL |
| 2025-02 | 57000.00 | -48500.00 |
| 2025-03 | 157000.00 | 100000.00 |
| 2025-04 | 108000.00 | -49000.00 |
| 2025-05 | 56000.00 | -52000.00 |

## Question 3: SQL Troubleshooting

### a. How an execution plan identifies expensive operations

An execution plan shows the step-by-step roadmap the database uses to run a query. It highlights expensive operations—such as Full Table Scans or Nested Loop Joins on large datasets—by showing high CPU usage, large memory consumption, and high estimated costs or row counts, allowing you to spot where bottlenecks occur.

### b. Index strategy for DepartmentID and Salary

- Recommendation: Create a composite (multi-column) index on (DepartmentID, Salary).
- Reasoning: Since queries filter by both columns, a composite index allows the database engine to quickly jump directly to the matching department and filter by salary within that subset in a single index scan, avoiding costly full table scans. Place DepartmentID first if it has higher selectivity (filters out more rows).

### c. SQL query to find duplicate customer email addresses

SQL

```
SELECT Email, COUNT(*) AS OccurrenceCount

FROM Customers

GROUP BY Email

HAVING COUNT(*) > 1;
```

# Question 4: Log Analysis

```
Last login: Wed Oct 28 09:29:28 on ttys000
aratihajgulkar@Adarsh-Hajgulkar ~ % cd downloads
aratihajgulkar@Adarsh-Hajgulkar downloads % mkdir -p ~/log-practice
aratihajgulkar@Adarsh-Hajgulkar downloads % cd ~/log-practice
aratihajgulkar@Adarsh-Hajgulkar log-practice % cat > application.log <<'EOF'
2026-10-28 09:10:23 INFO payment-service Started
2026-10-28 09:15:23 ERROR payment-service Transaction failed
2026-10-28 09:20:10 ERROR login-service Login failed
2026-10-28 10:05:12 INFO payment-service Processing
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:25:40 ERROR order-service Order failed
2026-10-28 10:40:00 ERROR login-service Authentication failed
2026-10-28 11:10:15 INFO payment-service Completed
2026-10-28 11:20:30 ERROR payment-service Refund failed
2026-10-28 11:45:50 ERROR order-service Payment timeout
2026-10-27 10:12:00 ERROR payment-service Old transaction failed
2026-10-27 12:15:00 INFO login-service Login successful
EOF
aratihajgulkar@Adarsh-Hajgulkar log-practice % cat application.log
2026-10-28 09:10:23 INFO payment-service Started
2026-10-28 09:15:23 ERROR payment-service Transaction failed
2026-10-28 09:20:10 ERROR login-service Login failed
2026-10-28 10:05:12 INFO payment-service Processing
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:25:40 ERROR order-service Order failed
2026-10-28 10:40:00 ERROR login-service Authentication failed
2026-10-28 11:10:15 INFO payment-service Completed
2026-10-28 11:20:30 ERROR payment-service Refund failed
2026-10-28 11:45:50 ERROR order-service Payment timeout
2026-10-27 10:12:00 ERROR payment-service Old transaction failed
```

```
2026-10-27 10:12:00 ERROR payment-service Old transaction failed
2026-10-27 12:15:00 INFO login-service Login successful
aratihajgulkar@Adarsh-Hajgulkar log-practice % grep "ERROR" application.log
2026-10-28 09:15:23 ERROR payment-service Transaction failed
2026-10-28 09:20:10 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:25:40 ERROR order-service Order failed
2026-10-28 10:40:00 ERROR login-service Authentication failed
2026-10-28 11:20:30 ERROR payment-service Refund failed
2026-10-28 11:45:50 ERROR order-service Payment timeout
2026-10-27 10:12:00 ERROR payment-service Old transaction failed
aratihajgulkar@Adarsh-Hajgulkar log-practice % grep "ERROR" application.log | tail -n 20
2026-10-28 09:15:23 ERROR payment-service Transaction failed
2026-10-28 09:20:10 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:25:40 ERROR order-service Order failed
2026-10-28 10:40:00 ERROR login-service Authentication failed
2026-10-28 11:20:30 ERROR payment-service Refund failed
2026-10-28 11:45:50 ERROR order-service Payment timeout
2026-10-27 10:12:00 ERROR payment-service Old transaction failed
aratihajgulkar@Adarsh-Hajgulkar log-practice % grep -c "ERROR" application.log
8
aratihajgulkar@Adarsh-Hajgulkar log-practice % grep "^2026-10-28.*ERROR" application.log > errors.txt
aratihajgulkar@Adarsh-Hajgulkar log-practice % cat errors.txt
2026-10-28 09:15:23 ERROR payment-service Transaction failed
2026-10-28 09:20:10 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:25:40 ERROR order-service Order failed
2026-10-28 10:40:00 ERROR login-service Authentication failed
2026-10-28 11:20:30 ERROR payment-service Refund failed
```

```
aratihajgulkar@Adarsh-Hajgulkar log-practice % grep -c "ERROR" application.log
8
aratihajgulkar@Adarsh-Hajgulkar log-practice % grep "^2026-10-28.*ERROR" application.log > errors.txt
aratihajgulkar@Adarsh-Hajgulkar log-practice % cat errors.txt
2026-10-28 09:15:23 ERROR payment-service Transaction failed
2026-10-28 09:20:10 ERROR login-service Login failed
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:25:40 ERROR order-service Order failed
2026-10-28 10:40:00 ERROR login-service Authentication failed
2026-10-28 11:20:30 ERROR payment-service Refund failed
2026-10-28 11:45:50 ERROR order-service Payment timeout
aratihajgulkar@Adarsh-Hajgulkar log-practice % wc -l < errors.txt
       7
aratihajgulkar@Adarsh-Hajgulkar log-practice % grep "ERROR" application.log | cut -c 1-13 | sort | uniq -c
   1 2026-10-27 10
   2 2026-10-28 09
   3 2026-10-28 10
   2 2026-10-28 11
aratihajgulkar@Adarsh-Hajgulkar log-practice % grep "ERROR" application.log | cut -c 1-13 | sort | uniq -c
   1 2026-10-27 10
   2 2026-10-28 09
   3 2026-10-28 10
   2 2026-10-28 11
aratihajgulkar@Adarsh-Hajgulkar log-practice % grep "ERROR" application.log | awk '{print $1, substr($2,1,2)}' | sort |
uniq -c
   1 2026-10-27 10
   2 2026-10-28 09
   3 2026-10-28 10
   2 2026-10-28 11
aratihajgulkar@Adarsh-Hajgulkar log-practice %
```

## Question 5: Permissions and System Troubleshooting

```
[aratihajgulkar@Adarsh-Hajgulkar ~ % sudo dseditgroup -o create finance
[aratihajgulkar@Adarsh-Hajgulkar ~ % dseditgroup -o read finance
 dsAttrTypeStandard:GeneratedUID -
                AC45D48A-FCE4-4F1D-B869-F88764A8ED1B
 dsAttrTypeStandard:PrimaryGroupID -
                501
 dsAttrTypeStandard:RecordName -
                finance
 dsAttrTypeStandard:RecordType -
                dsRecTypeStandard:Groups
 dsAttrTypeStandard:AppleMetaNodeLocation -
                /Local/Default
[aratihajgulkar@Adarsh-Hajgulkar ~ % mkdir -p ~/finance_app
[aratihajgulkar@Adarsh-Hajgulkar ~ % touch ~/finance_app/financial_report.txt
[aratihajgulkar@Adarsh-Hajgulkar ~ % echo "Confidential financial report" > ~/finance_app/financial_report.txt
[aratihajgulkar@Adarsh-Hajgulkar ~ % ls -l ~/finance_app/financial_report.txt
 -rw-r--r--  1 aratihajgulkar  staff  30 28 Oct 12:06 /Users/aratihajgulkar/finance_app/financial_report.txt
[aratihajgulkar@Adarsh-Hajgulkar ~ % sudo chown "$(whoami)" ~/finance_app/financial_report.txt
[aratihajgulkar@Adarsh-Hajgulkar ~ % sudo chgrp finance ~/finance_app/financial_report.txt
[aratihajgulkar@Adarsh-Hajgulkar ~ % chmod 640 ~/finance_app/financial_report.txt
[aratihajgulkar@Adarsh-Hajgulkar ~ % ls -l ~/finance_app/financial_report.txt
 -rw-r-----  1 aratihajgulkar  finance  30 28 Oct 12:06 /Users/aratihajgulkar/finance_app/financial_report.txt
[aratihajgulkar@Adarsh-Hajgulkar ~ % top -o cpu
[aratihajgulkar@Adarsh-Hajgulkar ~ % top -l 1 | head -n 10
 Processes: 531 total, 2 running, 1 stuck, 528 sleeping, 3153 threads
 2026/10/28 12:09:49
 Load Avg: 1.64, 1.80, 2.17
 CPU usage: 2.89% user, 13.62% sys, 83.47% idle
 SharedLibs: 266M resident, 53M data, 57M linkedit.
 MemRegions: 451857 total, 1499M resident, 109M private, 763M shared.
 PhysMem: 7542M used (1399M wired, 3277M compressor), 92M unused.
 VM: 278T vsize, 57G framework vsize, 166985(0) swapins, 361096(0) swapouts.
 Networks: packets: 3713510/4455M in, 680773/174M out.
 Disks: 5020507/133G read, 1516187/59G written.
[aratihajgulkar@Adarsh-Hajgulkar ~ % df -h
 Filesystem        Size    Used   Avail Capacity iused ifree %iused  Mounted on
 /dev/disk3s3s1   460Gi    13Gi   323Gi     4%    487k  3.4G    0%   /
 devfs            205Ki   205Ki     0Bi   100%     711     0  100%   /dev
 /dev/disk3s6     460Gi   4.0Gi   323Gi     2%       4  3.4G    0%   /System/Volumes/VM
 /dev/disk3s4     460Gi   9.1Gi   323Gi     3%    1.3k  3.4G    0%   /System/Volumes/Preboot
 /dev/disk3s2     460Gi    81Mi   323Gi     1%     113  3.4G    0%   /System/Volumes/Update
 /dev/disk1s2     500Mi   6.0Mi   481Mi     2%       3  4.9M    0%   /System/Volumes/xarts
 /dev/disk1s1     500Mi   6.0Mi   481Mi     2%      31  4.9M    0%   /System/Volumes/iSCPreboot
 /dev/disk1s3     500Mi   2.2Mi   481Mi     1%      46  4.9M    0%   /System/Volumes/Hardware
 /dev/disk3s1     460Gi   110Gi   323Gi    26%    845k  3.4G    0%   /System/Volumes/Data
 map auto_home      0Bi     0Bi     0Bi   100%       0     0     -   /System/Volumes/Data/home
 /dev/disk3s5     460Gi   1.4Gi   323Gi     1%      67  3.4G    0%   /Volumes/Recovery
[aratihajgulkar@Adarsh-Hajgulkar ~ % find ~/finance_app -type f -size +100M
[aratihajgulkar@Adarsh-Hajgulkar ~ % find /path/to/application -type f -size +100M
```

## Question 6: Bash Scripting

```
Last login: Wed Oct 28 12:07:13 on ttys003
aratihajgulkar@Adarsh-Hajgulkar ~ % mkdir -p ~/backup_practice
aratihajgulkar@Adarsh-Hajgulkar ~ % cd ~/backup_practice
aratihajgulkar@Adarsh-Hajgulkar backup_practice % nano backup_check.sh
aratihajgulkar@Adarsh-Hajgulkar backup_practice % cd ~/backup_practice
aratihajgulkar@Adarsh-Hajgulkar backup_practice % chmod +x backup_check.sh
aratihajgulkar@Adarsh-Hajgulkar backup_practice % ls -l backup_check.sh
-rwxr-xr-x  1 aratihajgulkar  staff  849 28 Oct 12:14 backup_check.sh
aratihajgulkar@Adarsh-Hajgulkar backup_practice % mkdir -p ~/backup_practice/mydata
aratihajgulkar@Adarsh-Hajgulkar backup_practice % echo "This is my important data" > ~/backup_practice/mydata/report.txt

aratihajgulkar@Adarsh-Hajgulkar backup_practice % ./backup_check.sh ~/backup_practice/mydata ~/backup_practice/backups
Backup successful: /Users/aratihajgulkar/backup_practice/backups/backup_20261028_121847.tar.gz
aratihajgulkar@Adarsh-Hajgulkar backup_practice % ls -lh ~/backup_practice/backups
total 8
-rw-r--r--  1 aratihajgulkar  staff   179B 28 Oct 12:18 backup_20261028_121847.tar.gz
aratihajgulkar@Adarsh-Hajgulkar backup_practice % ./backup_check.sh
Error: Please provide source and destination directories.
Usage: ./backup_check.sh <source_directory> <destination_directory>
aratihajgulkar@Adarsh-Hajgulkar backup_practice % ./backup_check.sh ~/wrong_folder ~/backup_practice/backups
Error: Source directory does not exist.
aratihajgulkar@Adarsh-Hajgulkar backup_practice % ls -lh ~/backup_practice/backups
total 8
-rw-r--r--  1 aratihajgulkar  staff   179B 28 Oct 12:18 backup_20261028_121847.tar.gz
aratihajgulkar@Adarsh-Hajgulkar backup_practice % tar -tzf ~/backup_practice/backups/backup_*.tar.gz
mydata/
mydata/report.txt
aratihajgulkar@Adarsh-Hajgulkar backup_practice %
```

# Question 7: PySpark DataFrame Operations

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count
from datetime import date

spark = SparkSession.builder.appName("EmployeeData").getOrCreate()

data = [
    (101, "Rahul", "IT", 75000, date(2022, 1, 15)),
    (102, "Priya", "HR", 55000, date(2021, 3, 10)),
    (103, "Amit", "IT", 62000, date(2023, 6, 20)),
    (104, "Sneha", "Finance", 68000, date(2020, 2, 12)),
    (105, "Kiran", "IT", 85000, date(2019, 9, 5)),
    (106, "Neha", "HR", 72000, date(2022, 7, 18)),
    (107, "Rohit", "Finance", 48000, date(2023, 4, 1)),
    (108, "Pooja", "HR", 61000, date(2022, 11, 11)),
    (109, "Vikas", "Finance", 90000, date(2020, 8, 25)),
    (110, "Anjali", "IT", 58000, date(2021, 12, 14)),
    (111, "Suresh", "Finance", 65000, date(2023, 5, 8)),
    (112, "Meera", "HR", 78000, date(2020, 10, 19)),
    (113, "Arjun", "IT", 66000, date(2022, 1, 24)),
    (114, "Kavya", "Finance", 52000, date(2023, 7, 17)),
    (115, "Manoj", "HR", 63000, date(2021, 5, 30))
    ]

columns = [
    "EmployeeID", "EmployeeName", "Department",
    "Salary", "JoiningDate"
]

df = spark.createDataFrame(data, columns)

df.show()
```

```
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|       101|       Rahul|        IT| 75000| 2022-01-15|
|       102|       Priya|        HR| 55000| 2021-03-10|
|       103|        Amit|        IT| 62000| 2023-06-20|
|       104|       Sneha|   Finance| 68000| 2020-02-12|
|       105|       Kiran|        IT| 85000| 2019-09-05|
|       106|        Neha|        HR| 72000| 2022-07-18|
|       107|       Rohit|   Finance| 48000| 2023-04-01|
|       108|       Pooja|        HR| 61000| 2022-11-11|
|       109|       Vikas|   Finance| 90000| 2020-08-25|
|       110|      Anjali|        IT| 58000| 2021-12-14|
|       111|      Suresh|   Finance| 65000| 2023-05-08|
|       112|       Meera|        HR| 78000| 2020-10-19|
|       113|       Arjun|        IT| 66000| 2022-01-24|
|       114|       Kavya|   Finance| 52000| 2023-07-17|
|       115|       Manoj|        HR| 63000| 2021-05-30|
+----------+------------+----------+------+-----------+
```

[9]:
```
df.printSchema()
df.show(5, truncate=False)
```

```
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)

+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|101       |Rahul       |IT        |75000 |2022-01-15 |
|102       |Priya       |HR        |55000 |2021-03-10 |
|103       |Amit        |IT        |62000 |2023-06-20 |
|104       |Sneha       |Finance   |68000 |2020-02-12 |
|105       |Kiran       |IT        |85000 |2019-09-05 |
+----------+------------+----------+------+-----------+
```

[10]:
```
df.filter(col("Salary") > 60000).show()
```

```
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|       101|       Rahul|        IT| 75000| 2022-01-15|
|       103|        Amit|        IT| 62000| 2023-06-20|
|       104|       Sneha|   Finance| 68000| 2020-02-12|
|       105|       Kiran|        IT| 85000| 2019-09-05|
|       106|        Neha|        HR| 72000| 2022-07-18|
|       108|       Pooja|        HR| 61000| 2022-11-11|
|       109|       Vikas|   Finance| 90000| 2020-08-25|
|       111|      Suresh|   Finance| 65000| 2023-05-08|
|       112|       Meera|        HR| 78000| 2020-10-19|
|       113|       Arjun|        IT| 66000| 2022-01-24|
|       115|       Manoj|        HR| 63000| 2021-05-30|
+----------+------------+----------+------+-----------+
```

[11]:
```
df = df.withColumn("AnnualSalary", col("Salary") * 12)
df.show()
```

```
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|       101|       Rahul|        IT| 75000| 2022-01-15|      900000|
|       102|       Priya|        HR| 55000| 2021-03-10|      660000|
|       103|        Amit|        IT| 62000| 2023-06-20|      744000|
|       104|       Sneha|   Finance| 68000| 2020-02-12|      816000|
|       105|       Kiran|        IT| 85000| 2019-09-05|     1020000|
|       106|        Neha|        HR| 72000| 2022-07-18|      864000|
|       107|       Rohit|   Finance| 48000| 2023-04-01|      576000|
|       108|       Pooja|        HR| 61000| 2022-11-11|      732000|
|       109|       Vikas|   Finance| 90000| 2020-08-25|     1080000|
|       110|      Anjali|        IT| 58000| 2021-12-14|      696000|
|       111|      Suresh|   Finance| 65000| 2023-05-08|      780000|
|       112|       Meera|        HR| 78000| 2020-10-19|      936000|
|       113|       Arjun|        IT| 66000| 2022-01-24|      792000|
|       114|       Kavya|   Finance| 52000| 2023-07-17|      624000|
```

[12]:
```
df.groupBy("Department").agg(
    avg("Salary").alias("AverageSalary"),
    count("*").alias("EmployeeCount")
).show()
```

```
+----------+-------------+-------------+
|Department|AverageSalary|EmployeeCount|
+----------+-------------+-------------+
|        IT|      69200.0|            5|
|        HR|      65800.0|            5|
|   Finance|      64600.0|            5|
+----------+-------------+-------------+
```

[13]:
```
df.orderBy(col("Salary").desc()).show(5)

spark.stop()
```

```
+----------+------------+----------+------+-----------+------------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+------------+----------+------+-----------+------------+
|       109|       Vikas|   Finance| 90000| 2020-08-25|     1080000|
|       105|       Kiran|        IT| 85000| 2019-09-05|     1020000|
|       112|       Meera|        HR| 78000| 2020-10-19|      936000|
|       101|       Rahul|        IT| 75000| 2022-01-15|      900000|
|       106|        Neha|        HR| 72000| 2022-07-18|      864000|
+----------+------------+----------+------+-----------+------------+
only showing top 5 rows
```

[ ]:

---

### Question 8: Data Cleaning and Aggregation (7 Marks)

A company receives a CSV file named transactions.csv containing:

Answers:

a) schema = StructType([

StructField("TransactionID", StringType(), True),

StructField("CustomerID", StringType(), True),

StructField("TransactionDate", StringType(), True),

StructField("Product", StringType(), True),

StructField("Quantity", StringType(), True),

StructField("UnitPrice", StringType(), True)

])

df = spark.read.csv("transactions.csv", header=True, schema=schema)

b). df = df.dropDuplicates([

"CustomerID", "TransactionDate", "Product",

```
    "Quantity", "UnitPrice"
])
c.  df = df.dropna(subset=[
    "TransactionID", "CustomerID", "TransactionDate",
    "Product", "Quantity", "UnitPrice"
])
d. from pyspark.sql.functions import col, to_date, coalesce
df = df.withColumn(
    "TransactionDate",
    coalesce(
        to_date(col("TransactionDate"), "yyyy-MM-dd"),
        to_date(col("TransactionDate"), "dd-MM-yyyy")
    )
)
df = df.dropna(subset=["TransactionDate"])
e.  df = df.withColumn(
    "Revenue",
    col("Quantity").cast("int") * col("UnitPrice").cast("double")
)
f. from pyspark.sql.functions import sum as spark_sum
df.groupBy("Product").agg(
    spark_sum("Revenue").alias("TotalRevenue")
).orderBy(
    col("TotalRevenue").desc()
).show()
```

## Question 9: PySpark Joins and Performance (7 Marks)

Answers:

a) from pyspark.sql import SparkSession
from pyspark.sql.functions import sum as spark_sum, broadcast
spark = SparkSession.builder.appName("CustomerOrders").getOrCreate()
customers = spark.read.csv("customers.csv", header=True, inferSchema=True)
orders = spark.read.csv("orders.csv", header=True, inferSchema=True)

b)joined_df = orders.join(customers, on="CustomerID", how="inner")
joined_df.show()

c) customers_without_orders = customers.join(
    orders, on="CustomerID", how="left_anti"
)
customers_without_orders.show()

d)joined_df.groupBy("City").agg(
    spark_sum("OrderAmount").alias("TotalOrderValue")
).show()

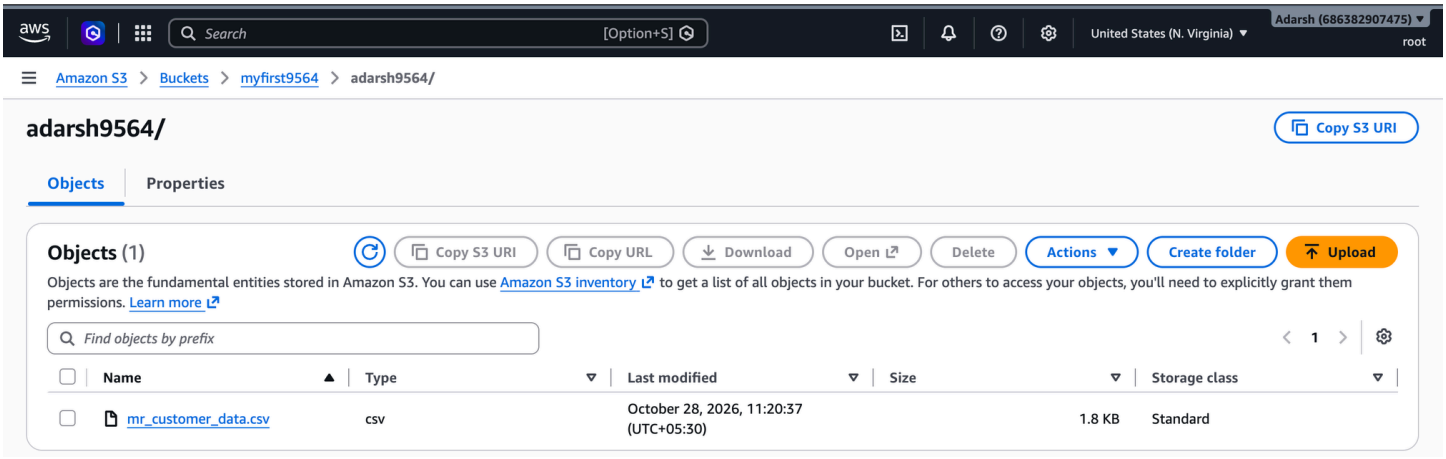
e)joined_df = orders.join(broadcast(customers), on="CustomerID")

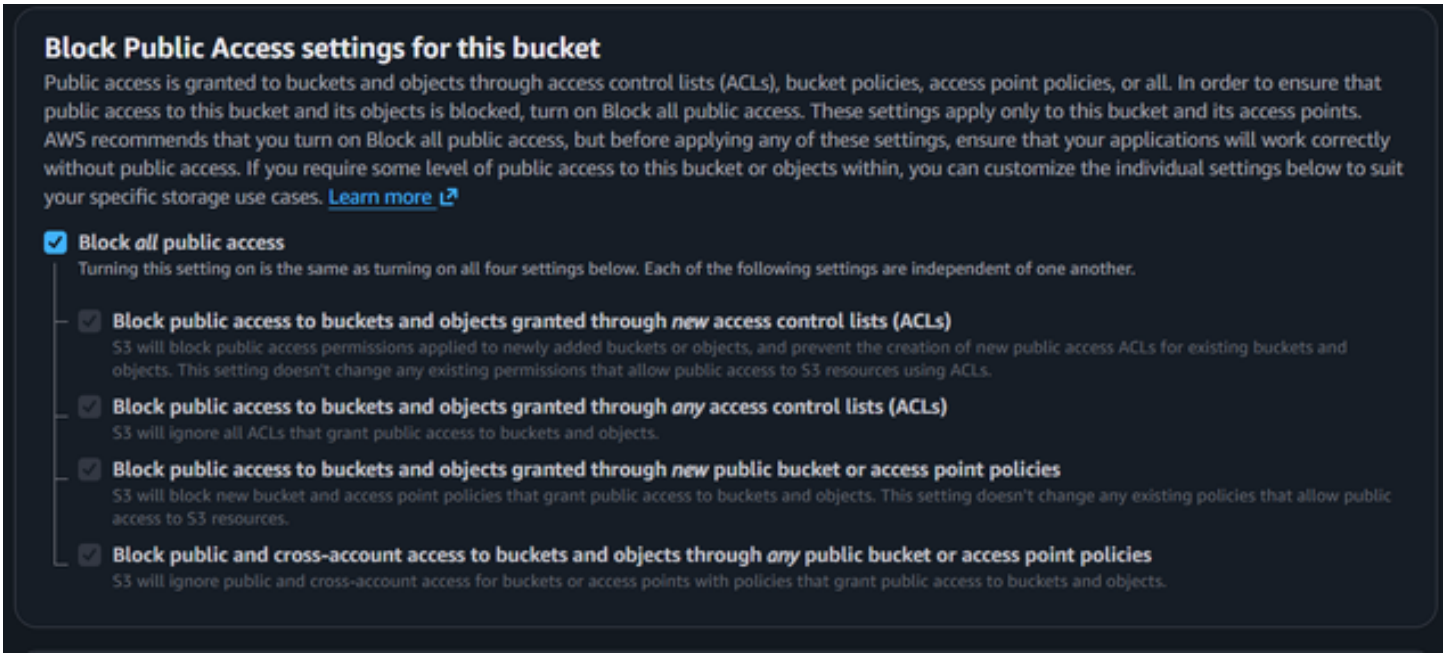
f)  f iltered = orders.filter(orders.OrderAmount > 1000)
filtered.show()

# SECTION D: AWS CLOUD SERVICES

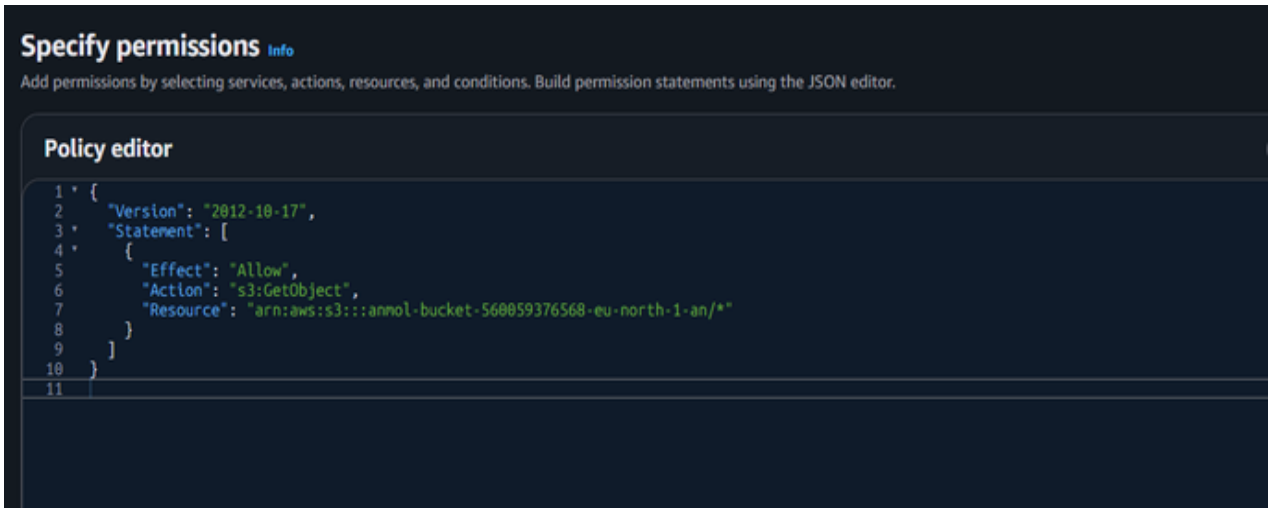
### Question 10: S3 and IAM Security

a. Create an S3 bucket with Block Public Access enabled and upload a sample file.






b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets. (2 marks)




c. Explain how you would verify that an unauthorized user cannot read the file. (1 mark)
answer: If an unauthorized user tries to access the file, an Access Denied (403) error will occur. We can verify the failed access attempt using AWS CloudTrail logs (after enabling S3 data events).

d. Explain how encryption at rest protects the stored data. (1 mark)
answer: Encryption at rest protects stored data by encrypting it using encryption keys. Even if an unauthorized person gains access to the storage, they cannot read the data without the required decryption key. In AWS S3, this can be implemented using Server-Side Encryption (SSE).

**Question 11: EC2 Troubleshooting and CloudWatch**

a. Check the EC2 instance state and status checks.
Open the AWS EC2 console and check whether the instance is in the Running state. Verify that both system status checks and instance status checks are passing.

b. Explain how security groups and route tables can affect application connectivity.
Security groups control incoming and outgoing traffic to the EC2 instance. Incorrect rules may block application traffic. Route tables determine where network traffic is sent, and incorrect routes can prevent users from connecting to the application.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.
Open Amazon CloudWatch and create an alarm for the EC2 instance's CPUUtilization metric. Set the threshold to greater than 85% and define the evaluation period. Configure an SNS notification to alert the administrator when the alarm is triggered.

d. Identify one method for investigating the root cause of high CPU utilization.
Use the EC2 instance's operating system tools, such as the top command in Linux, to identify processes consuming high CPU resources.

**Question 12: Event-Driven Processing**
**A company uploads CSV files to an S3 bucket. Each uploaded file must be validated and processed automatically, with the results stored in another location. Design a solution using AWS services.**

**a. Identify a suitable service for executing the processing code. (1 mark)**
AWS Lambda is suitable for executing code to validate and process uploaded CSV files automatically.

**b. Explain how an S3 upload can trigger processing. (1 mark)**
Configure an S3 event notification to trigger the Lambda function whenever a new CSV file is uploaded to the input bucket.

**c. Explain the IAM permissions needed to read the input file and write the results. (1 mark)**
Assign an IAM execution role to Lambda with s3:GetObject permission for the input bucket and s3:PutObject permission for the output bucket.

**d. Explain how errors and processing logs can be captured. (1 mark)**
Use Amazon CloudWatch Logs to store Lambda execution logs and errors. CloudWatch can also be used to monitor failures.

**e. Explain how to prevent duplicate processing or recursive triggering.**
Use a separate S3 bucket for the processed results so that output uploads do not trigger the same Lambda function. To prevent duplicate processing, make the function idempotent by tracking processed files.

**Question 13: State Machine Design (7 Marks)**

**a. Identify suitable states for validation, processing, storage, and notification. (2 marks)**

Use AWS Step Functions with the following states: Task states for validation, data transformation, sales calculation, S3 storage, and sending the completion notification. Use Choice to check whether the input file is valid and Fail to handle invalid files.

**b. Explain how a Choice state can route invalid files to a failure path and valid files to processing. (1 mark)**

The Choice state checks the validation result. If the file is valid, it moves to processing. If the file is invalid, it routes the workflow to a Fail state.

**c. Explain how Retry and Catch can handle transient failures and permanent errors. (2 marks)**

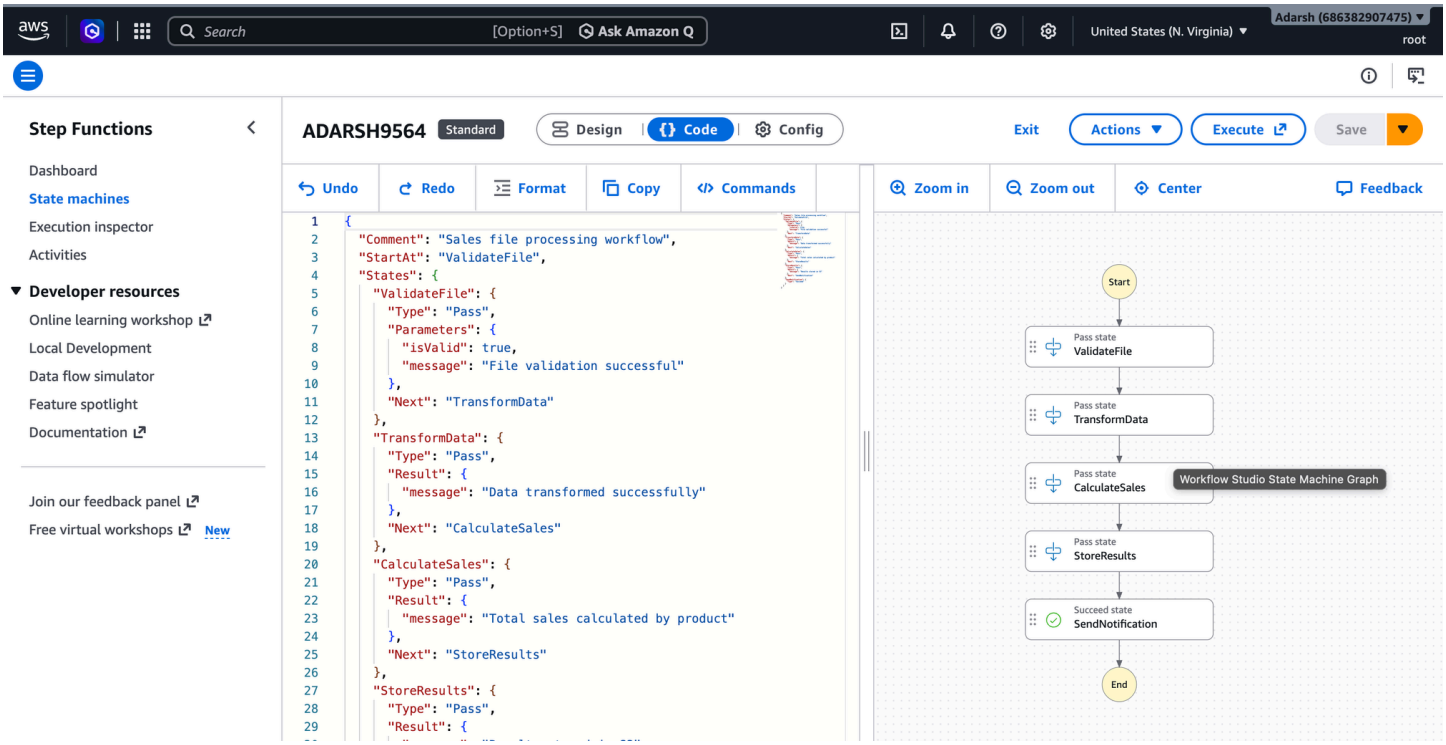
Retry automatically retries temporary failures, such as service errors. Catch handles errors that cannot be recovered by retrying and routes the workflow to an error-handling state or Fail state.

**d. Explain how the state machine can invoke Lambda functions or other supported AWS services. (1 mark)**

Step Functions can invoke AWS Lambda functions using Task states to validate, transform, and calculate sales. It can also use supported AWS service integrations to store results in S3 and send notifications through Amazon SNS.

**e. Explain how execution history helps troubleshoot a failed workflow. (1 mark)**

Step Functions execution history shows each state's input, output, status, and error details. This helps identify the failed state and troubleshoot the problem.



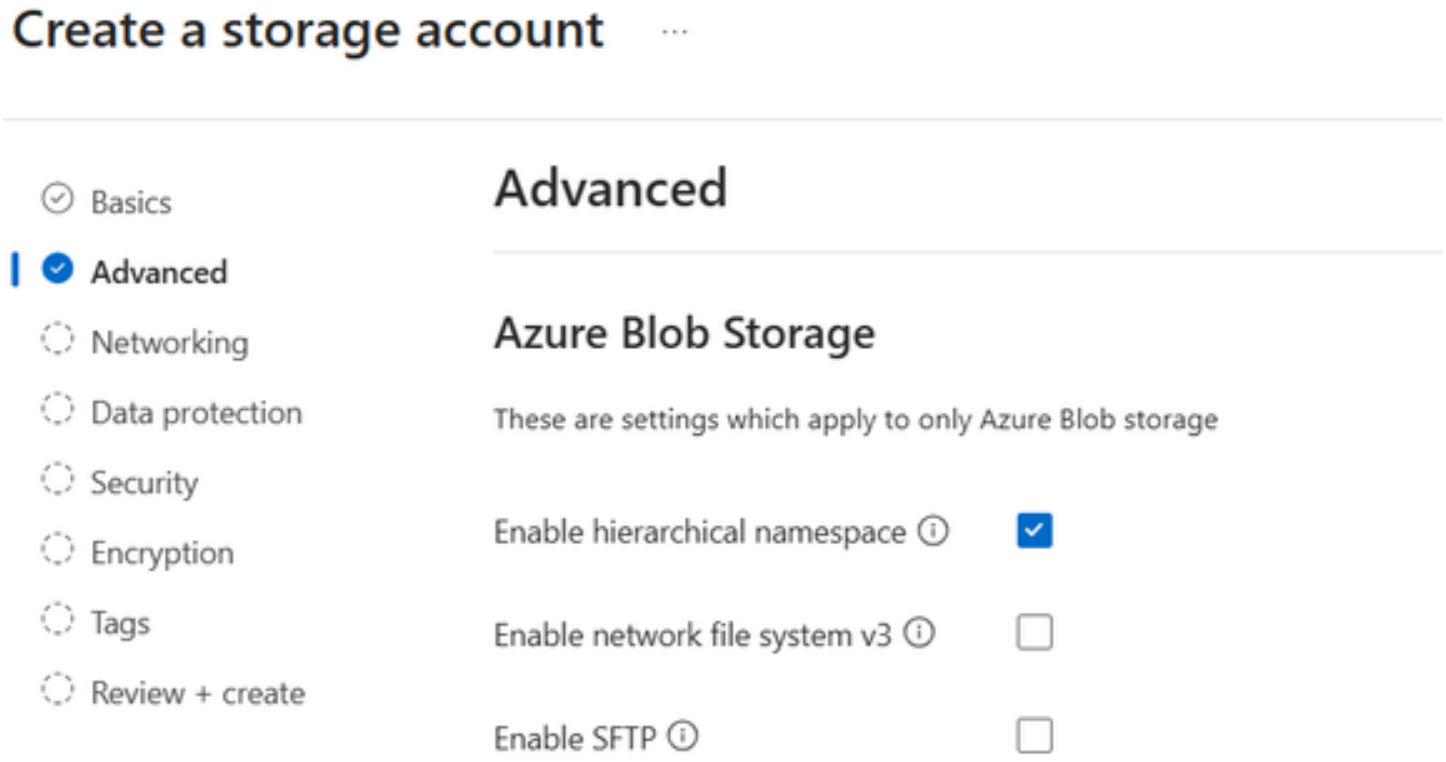
Question 14: Step Functions Implementation (8 Marks)
A data engineering team has developed three processing functions:
• ValidateFile
• TransformData
• LoadResults
The functions must execute in sequence. Requirements:
• If validation fails, the workflow must stop.
• If transformation fails temporarily, the workflow must retry.
• If loading fails permanently, the workflow must record the failure. Tasks:
a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.

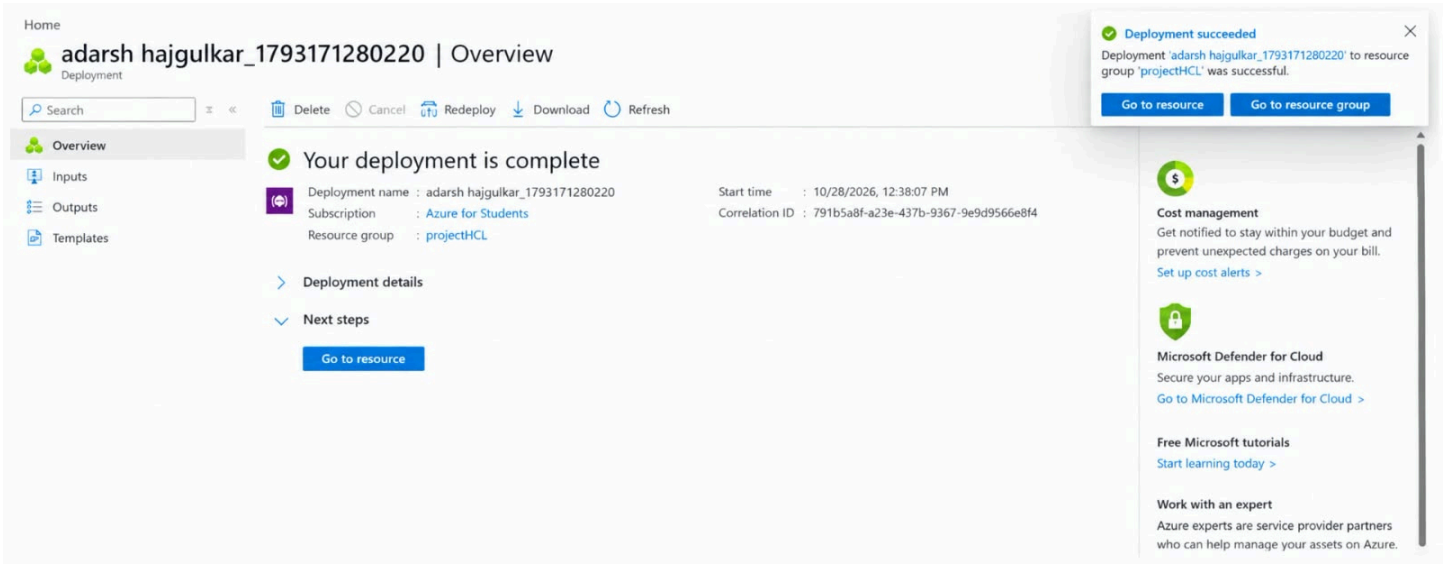
```
{
  "Comment": "Data Processing Workflow",
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "Lambda ARN: ValidateFile",
      "Next": "TransformData"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "Lambda ARN: TransformData",
      "Retry": [
        {
          "ErrorEquals": [
            "Lambda.ServiceException"
          ],
          "IntervalSeconds": 2,
          "MaxAttempts": 3,
          "BackoffRate": 2
        }
      ],
      "Next": "LoadResults"
    },
    "LoadResults": {
      "Type": "Task",
      "Resource": "Lambda ARN: LoadResults",
      "Catch": [
        {
          "ErrorEquals": [
            "States.ALL"
          ],
          "Next": "HandleError"
        }
      ],
      "End": true
    },
    "HandleError": {
      "Type": "Pass",
      "Result": "Loading failed",
      "End": true
    }
  }
}
```

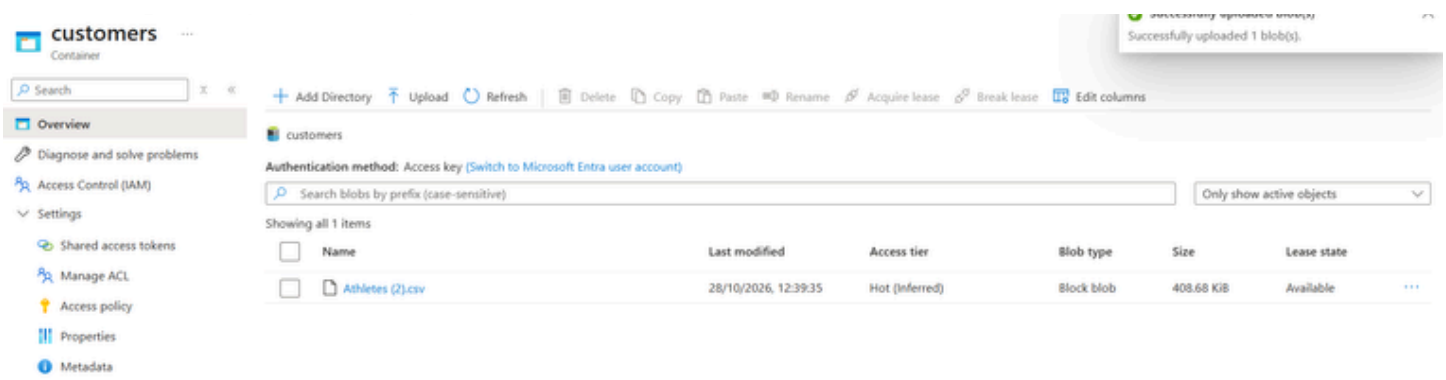
**b. Configure a retry policy for the transformation task. (2 marks)**
The Retry state retries temporary errors such as Lambda.ServiceException. It waits for 2 seconds, retries up to 3 times, and doubles the wait time after each retry using BackoffRate: 2.
**c. Configure a Catch transition for the loading task. (1 mark)**
The Catch block captures loading errors and transfers control to the HandleError state, where the failure is recorded.
**d. Explain how data from one task is passed to the next task. (1 mark)**
Step Functions passes the output of one task as the input to the next task by default. JSON data can be used to share processing results between functions.
**e. Explain how you would test successful and failed executions. (1 mark)**
Run the workflow with valid input to verify successful completion. Test with invalid input and simulated transformation or loading errors to verify that retries and error handling work correctly.

**Question 15: Azure Data Lake Storage and Security**
**a. Configure a storage account with hierarchical namespace enabled. (1 mark)**

Open the Azure Portal and create a Storage Account. Select the appropriate subscription and resource group, then enable the Hierarchical namespace option to use Azure Data Lake Storage Gen2.




Open the storage account, go to Containers, create a container, and upload a sample CSV dataset for analytics.

**c. Explain how Microsoft Entra ID and Azure RBAC manage access to the data. (2 marks)**

Microsoft Entra ID manages user identities and authentication. Azure RBAC assigns permissions to users or groups based on their roles, such as Storage Blob Data Reader or Storage Blob Data Contributor, to control access to the data.

**d. Explain how you would verify that unauthorized users cannot access the dataset. (1 mark)**

Sign in as a user without the required permissions and attempt to access or download the dataset. Verify that access is denied, and check the Azure Monitor or diagnostic logs if configured

**Question 16: Azure VM Troubleshooting**

A virtual machine is running in Azure, but administrators cannot connect to it.

Perform or document the following:

a. Verify the VM's provisioning and power states.

Answer:

Check the VM's status in the Azure portal under Virtual Machines. Verify that the provisioning state is Succeeded and the power state is Running.

b. Explain how Network Security Group rules can prevent connectivity.

Answer:

Network Security Group (NSG) rules may block incoming connections if the required port, such as SSH (22) or RDP (3389), is denied. Verify that the inbound rules allow traffic from authorized sources.

c. Explain how subnet and routing configuration affect connectivity.

Answer:

Incorrect subnet configurations, route tables, or user-defined routes may prevent traffic from reaching the VM or returning to the client. Check the subnet association, route table, and network connectivity.

d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet.

Answer:

Use Azure Bastion to connect securely to the VM through the Azure portal without assigning a public IP address to the VM. Alternatively, use Azure Serial Console to troubleshoot supported boot and connectivity issues. Restrict NSG rules to authorized sources and avoid opening ports to the entire internet.

**Question 17: Azure Data Integration**

**A company receives a CSV file containing daily customer transactions and needs to prepare it for analysis. Design a solution using Azure services.**

**a. Select an appropriate service to orchestrate data movement.**

Check the VM's status in the Azure portal under Virtual Machines. Verify that the provisioning state is Succeeded and the power state is Running.

**b. Explain how to copy the CSV file into Azure Data Lake Storage Gen2. (1 mark)**

Network Security Group (NSG) rules may block incoming connections if the required port, such as SSH (22) or RDP (3389), is denied. Verify that the inbound rules allow traffic from authorized sources.

**c. Describe how to organize files by ingestion date. (1 mark)**

Incorrect subnet configurations, route tables, or user-defined routes may prevent traffic from reaching the VM or returning to the client. Check the subnet association, route table, and network connectivity.

**d. Explain how to handle malformed records and missing values. (1 mark) e. Explain how to monitor pipeline execution and validate data completeness. (1 mark)**

Use Azure Bastion to connect securely to the VM through the Azure portal without assigning a public IP address to the VM. Alternatively, use Azure Serial Console to troubleshoot supported boot and connectivity issues. Restrict NSG rules to authorized sources and avoid opening ports to the entire internet.